```
work>
```

The command has created a directory named `demo`. Pop down into that directory and list its contents (using `ls` on a Unix box or using `dir` on Windows). You should see a bunch of files and subdirectories:

```
work> cd demo
demo> ls -p
Gemfile         app/            db/             storage/
Gemfile.lock    bin/            lib/            test/
README.md       config/         log/            tmp/
Rakefile        config.ru       public/         vendor/
```

All these directories (and the files they contain) can be intimidating to start with, but you can ignore most of them for now. In this chapter, we'll only use two of them directly: the `bin` directory, where we'll find the Rails executables, and the `app` directory, where we'll write our application.

Examine your installation using the following command:

```
demo> bin/rails about
```

Windows users need to prefix the command with `ruby` and use a backslash:

```
demo> ruby bin\rails about
```

If you get a Rails version other than `7.0.4`, reread *Choosing a Rails Version*.

This command also detects common installation errors. For example, if it can't find a JavaScript runtime, it provides you with a link to available runtimes.

As you can see from the `bin/` prefix, this is running the `rails` command from the `bin` directory. This command is a wrapper, or *binstub*, for the Rails executable. It serves two purposes: it ensures that you're running with the correct version of every dependency, and it speeds up the startup times of Rails commands by preloading your application.

If you see a bunch of messages concerning already initialized constants or a possible conflict with an extension, consider deleting the `demo` directory, creating